

Peterson's Algorithm in Why3

Monolithic Proofs and Refinement for Mutual Exclusion

Miguel Carvalho (PG61153) ■ Margarida Pires (PG61130)

2-process Peterson's Algorithm $\rightarrow$ N-process Filter algorithm $\rightarrow$ Refinement

Goal and project structure

Objective. Verify mutual exclusion for Peterson-style algorithms in Why3.

Main property

```
forall p q.  
  pc[p] = Critical ->  
  pc[q] = Critical ->  
  p = q
```

Repository

```
src/  
  lib/  
    inductiveness.mlw  
    refinement.mlw  
  monolithic/  
    2_processes.mlw  
    N_processes.mlw  
    N_processes_split_attempt.mlw  
  refinement/  
    mutual_exclusion_spec.mlw  
    filter_abstract.mlw  
    filter_concrete_split.mlw
```

Background

Peterson's algorithm is a mutual exclusion algorithm for shared-memory concurrency.

It does **not** use a lock or mutex object. Instead, it uses shared variables:

```
turn   : 0..1  
want0  : bool  
want1  : bool
```

The process trying to enter the critical section:

- announces interest with `want`;
- gives priority to the other process with `turn`;
- busy-waits until the guard is true.

Two-process Peterson algorithm

Variables

```
var
    turn  : 0..1;
    want0 : boolean = false;
    want1 : boolean = false;
```

process0

```
procedure process0;
begin
    while true do begin
        0:      (* idle *)
        1:      want0 := true;
        2:      turn  := 1;
        3:      repeat until (not want1 or turn = 0);
        4:      (* critical region *)
        5:      want0 := false;
    end
end;
```

process1

```
procedure process1;
begin
    while true do begin
        0:      (* idle *)
        1:      want1 := true;
        2:      turn  := 0;
        3:      repeat until (not want0 or turn = 1);
        4:      (* critical region *)
        5:      want1 := false;
    end
end;
```

2-process Why3 model

Control states

```
type process = P0 | P1
```

```
type state =  
  Zero | One | Two |  
  Three | Four | Five
```

Shared variables

```
type world = {  
  pc    : map process state;  
  want  : map process bool;  
  turn  : process;  
}
```

Label meaning

Zero	idle
One	set want [p]
Two	set turn
Three	wait guard
Four	critical section
Five	reset want [p]

Result

```
goal correctness:  
  forall w. reachable w ->  
    atMostOneCS w
```

2-process invariant idea

The property `atMostOneCS` is true, but it is not strong enough by itself.

The inductive invariant records two facts:

1. when a process is trying or inside the critical section, `want[p]` is true;

```
forall p:process.  
  interested w p -> w.want[p]
```

2. the waiting process cannot enter while the other is already in the critical section.

```
forall p:process.  
  w.pc[p] = Three /\ w.pc[other p] = Four ->  
  w.turn = other p
```

This gives the standard proof shape:

$$reachable(w) \Rightarrow inv(w) \Rightarrow atMostOneCS(w)$$

From 2 processes to N: Filter algorithm

For N processes, Peterson's direct two-process structure is replaced by levels.

```
var
    level  : array [0..N-1] of integer = [0, ..., 0];
    victim : array [1..N-1] of integer;

procedure process(i);
begin
    while true do begin
0:      (* idle *)
        for l = 1 to N-1 do begin
1:          level[i] := l;
2:          victim[l] := i;
3:          repeat until
                forall k. k != i ->
                    level[k] < l or victim[l] != i;
            end;
4:          (* critical region *)
5:          level[i] := 0;
        end
    end;
end;
```

N-process model in Why3

Processes and levels

```
type process = int
type level = int

predicate valid_process p =
  0 <= p < n_processes

predicate valid_level l =
  1 <= l < n_processes
```

Filter variables

```
type world = {
  pc      : map process state;
  lvl     : map process level;
  victim  : map level process;
}
```

Waiting guard

```
predicate waiting_guard (w:world) (p:process) =
  forall q:process.
    valid_process q -> q <> p ->
    w.lvl[q] < w.lvl[p] /\ w.victim[w.lvl[p]] <> p
```


Counting invariant for the Filter algorithm

The key N-process invariant is global and level-indexed.

```
predicate filter_bound (w:world) =  
  forall l:level.  
    valid_level l ->  
      count_passed_level w l n_processes <= n_processes - l
```

Interpretation:

At level l , at most $N - l$ processes can have passed that level.

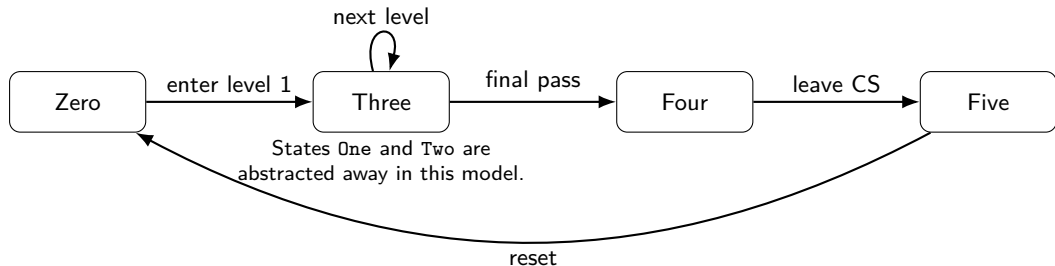
At the final level:

$$l = N - 1 \quad \Rightarrow \quad N - l = 1$$

Therefore, at most one process can pass the final level, so at most one process can be in the critical section.

Proof-friendly N-process abstraction

The proved monolithic N-process model uses an atomic level-entry abstraction.



This combines:

`level[p] := 1;` and `victim[l] := p;`

into one proof-friendly transition.

Monolithic proof results

Two-process Peterson

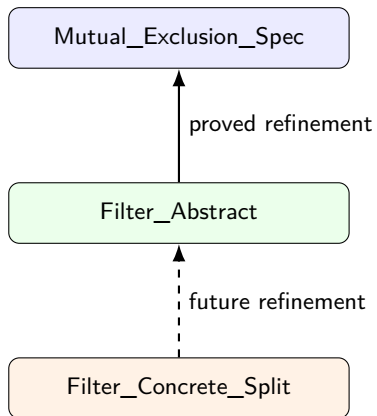
```
goal correctness:  
  forall w:world.  
    reachable w -> atMostOneCS w
```

N-process abstract Filter model

```
goal correctness:  
  forall w:world.  
    reachable w -> atMostOneCS w
```

The N-process proof relies on the counting invariant `filter_bound` and a lemma showing that two processes counted at the final level must be equal.

Refinement hierarchy



Most abstract model: mutual exclusion spec

The top-level specification tracks only the control state:

```
type state = Idle | Trying | CS | Exit

type world = {
  pc : map process state;
}
```

It does not mention Peterson variables, levels, victims, etc.

The invariant is just the desired safety property:

```
predicate inv (w:world) =
  atMostOneCS w
```

A process may enter CS only when no other process is already in CS.

Filter abstraction refines the spec

The Filter abstraction introduces implementation variables:

```
type world = {  
  fpc      : map process Mutual_Exclusion_Spec.state;  
  flvl     : map process level;  
  fvictim  : map level process;  
}
```

The refinement map hides lvl and victim:

```
let ghost function refn (w:world)  
  : Mutual_Exclusion_Spec.world =  
  { pc = w.fpc }
```

So the abstract specification sees only whether a process is idle, trying, in the critical section, or exiting.

Concrete split-step model and future work

The concrete split-step model is defined as the next refinement target:

`IdleC -> SetLevel -> SetVictim -> TryingC -> CSC -> ExitC -> IdleC`

It separates the doorway instructions:

```
level[i]  := 1
victim[l] := i
```

Open proof obligation:

- define a refinement map hiding `SetLevel` and `SetVictim`;
- abstract partially updated `lv1` and `victim`;
- prove the counting invariant remains inductive for the fully split model.

Conclusion

- Proved mutual exclusion for the original 2-process Peterson algorithm.
- Generalized to the N-process Filter algorithm using a proof-friendly abstraction.
- Used the standard counting invariant: at level l , at most $N - l$ processes can pass.
- Built a refinement hierarchy:

`Filter_Abstract` refines `Mutual_Exclusion_Spec`

- Included the concrete split-step Filter model as the next refinement target.
- Related work: [A Proof of Peterson's Algorithm \(in Coq\) by James Wilcox](#).

Final result: Why3 proves mutual exclusion for the completed models.